Two Hundred Questions in Formal Methods: Traditional Approaches and Deppy's Analogues

Generated for the deppy repository

April 7, 2026

Contents

Preface 3

1 Foundations and semantic viewpoints 4

2 Specifications, contracts, and type structure 10

3 Abstract interpretation and static analysis 16

4 Symbolic methods, solvers, and model checking 22

5 Memory, separation, and concurrency reasoning 28

6 Equivalence, refinement, and program transformation 34

7 Temporal reasoning, termination, and reactive properties 40

8 Proof assistants, synthesis, and automation 46

9 Security, robustness, and policy reasoning 52

10 Methodology, diagnosis, and the specifically deppy viewpoint 58

Closing note 64

Preface

This document is an incremental catalogue of 200 comparisons between basic formal-methods topics and deppy's sheaf-cohomological approach. For each topic, the document gives three things: the traditional method in its own terms, the closest honest deppy analogue, and a relationship label that says whether the comparison is best read as a generalization, a direct instance, an asymptotic improvement, an orthogonal complement, or only a partial analogue.

The goal is not to force every topic into the same mold. In some places deppy is clearly a direct reframing, as with obstruction-style bug detection and cover-relative equivalence. In others it is better understood as a manager of heterogeneous evidence, or as a retrofit semantic checker that is weaker than construction-time type disciplines but broader in applicability to ordinary Python.

The recurring deppy idea is this: local facts live at observation sites, restriction maps transport them, globally consistent families form zero-th cohomology, and irreducible failures of local-to-global composition appear as first-cohomology obstructions. That perspective is sometimes a strict extension of traditional machinery, sometimes merely a compatible overlay, and occasionally not yet the right tool at all.

Chapter 1

Foundations and semantic viewpoints

These entries compare the classic semantic lenses of formal methods with deppy's site-and-presheaf view of programs.

1. How does deppy compare to Hoare logic?

Traditional approach. Hoare logic organizes proofs around triples of the form {P} C {Q}, so the central question is whether a command transforms states satisfying a precondition into states satisfying a postcondition.

Deppy analogue. Deppy keeps the local-obligation spirit but relocates it to observation sites. Preconditions and postconditions become local sections and restriction maps, and failure to compose those local facts globally appears as a first-cohomology obstruction.

Relationship. Relationship: semantic recasting and partial generalization. Hoare triples become one way of describing local gluing constraints inside a larger sheaf-shaped object.

2. How does deppy compare to weakest preconditions?

Traditional approach. Weakest precondition calculi push obligations backward from a desired postcondition until one gets the weakest input condition that guarantees it.

Deppy analogue. Deppy also pushes requirements backward, but through presheaf restriction maps rather than a single transformer. The interesting output is not only the propagated requirement, but whether the propagated family can be glued without contradiction.

Relationship. Relationship: direct analogue. Restriction maps play the role of backward predicate propagation, while cohomology classifies the residual mismatch.

3. How does deppy compare to strongest postconditions?

Traditional approach. Strongest postconditions push facts forward from the input state to characterize everything reachable after execution.

Deppy analogue. Deppy has an analogous forward face in its globally compatible sections. What it adds is a dual global summary: not only which facts survive forward propagation, but which disagreements remain unavoidable after all local propagation has been accounted for.

Relationship. Relationship: paired analogue. Global sections resemble strongest-postcondition consistency, and first cohomology records what strongest postconditions alone do not summarize.

4. How does deppy compare to operational semantics?

Traditional approach. Operational semantics explains a program by step-by-step transition rules, either small-step or big-step, and proofs usually reason over those transitions directly.

Deppy analogue. Deppy does not replace transitions with magic; it abstracts them into data-flow morphisms between sites. The semantic burden shifts from enumerating all transitions to understanding whether local facts transported along those morphisms agree on overlaps.

Relationship. Relationship: abstraction over operational detail. Operational rules are the substrate from which deppy builds its site category.

5. How does deppy compare to denotational semantics?

Traditional approach. Denotational semantics maps programs into mathematical objects whose equations capture behavior compositionally.

Deppy analogue. Deppy is denotational in spirit because it assigns each program a semantic presheaf, plus associated Cech data. The target object is not just a function or domain element, but a structured local-to-global artifact whose cohomology exposes bugs and equivalence failures.

Relationship. Relationship: genuine generalization of the denotational stance. The denotation is enriched with locality, overlaps, and obstruction theory.

6. How does deppy compare to axiomatic semantics?

Traditional approach. Axiomatic semantics reasons syntactically from proof rules rather than by executing or simulating the program.

Deppy analogue. Deppy is close in attitude because it proves obligations from local rules at sites, but it packages those rules into a geometric object. Instead of asking only whether each rule application is locally justified, it asks whether all the locally justified pieces can coexist globally.

Relationship. Relationship: compositional lift. Axiomatic rules become local proof data inside a richer gluing problem.

7. How does deppy compare to big-step semantics?

Traditional approach. Big-step semantics jumps directly from initial states to final results, which is concise but tends to hide intermediate program structure.

Deppy analogue. Deppy needs intermediate structure because its sites live at boundaries, guards, calls, and error points. It is therefore less like a pure big-step account and more like a systematic

decomposition of the intermediate observations that big-step rules usually compress away.

Relationship. Relationship: partial analogue. Big-step gives endpoints; deppy insists on the overlaps between those endpoints.

8. How does deppy compare to small-step semantics?

Traditional approach. Small-step semantics exposes each reduction and is well suited for reasoning about control, divergence, and concurrency.

Deppy analogue. Deppy does not track every reduction step. Instead it chooses a cover of semantically meaningful observation sites and reasons over the compatibility of facts at those sites. This can be much more compact when the question is about global consistency rather than full trace structure.

Relationship. Relationship: abstraction and compression. Deppy is usually coarser than small-step semantics, but more structured than an arbitrary summary.

9. How does deppy compare to syntax-directed induction proofs?

Traditional approach. Many basic proofs in formal methods proceed by induction on the syntax of terms or commands, following the grammar of the language.

Deppy analogue. Deppy still relies on syntax to synthesize its sites and morphisms, so syntax remains the entry point. The difference is that after the syntax-induced decomposition is built, the main invariants are linear-algebraic and topological rather than purely inductive.

Relationship. Relationship: compatible but not identical. Syntax-directed structure feeds the cover; cohomology then reasons across the resulting overlaps.

10. How does deppy compare to logical-relations proofs?

Traditional approach. Logical relations prove semantic properties by relating terms across types, worlds, or program contexts in a recursively defined relation.

Deppy analogue. Deppy's closest analogue is its isomorphism presheaf for equivalence: each aligned site asks for a local relation between implementations, and the global verdict depends on whether those local relations glue. It is less expressive for higher-order meta-theory, but more operational for concrete code comparison.

Relationship. Relationship: partial analogue. Deppy borrows the local relation idea, then adds a geometric gluing criterion.

11. How does deppy compare to abstract machines?

Traditional approach. Abstract machines such as CEK or SECD expose evaluation states explicitly and let proofs track how stacks, environments, and continuations evolve.

Deppy analogue. Deppy usually stands one layer above that granularity. It can treat machine states as sources of local facts, but its native object is the presheaf over observation sites, not the

machine configuration graph itself.

Relationship. Relationship: implementation substrate. Abstract machines can feed deppy, but they are not themselves the analogue.

12. How does deppy compare to control-flow graphs?

Traditional approach. Control-flow graphs make program structure explicit as nodes and edges, enabling standard dataflow analyses and reachability questions.

Deppy analogue. Deppy's site category is wiser than a plain CFG in one specific way: nodes are semantic observation sites, and overlaps matter as much as edges. A CFG says where control may go; deppy asks what local sections on overlapping regions can be glued as control and data flow interact.

Relationship. Relationship: strict enrichment. A CFG is one shadow of the structure from which deppy builds a cover.

13. How does deppy compare to SSA form?

Traditional approach. SSA names each assignment uniquely so that def-use reasoning and solver encoding become cleaner.

Deppy analogue. Deppy benefits from SSA-like simplification because local sections become easier to phrase when values have single assignment identities. But SSA is a representation trick, whereas deppy's distinctive move is to turn those local value facts into a presheaf and compute obstruction classes.

Relationship. Relationship: enabling representation, not a competing theory. SSA makes the local pieces cleaner; deppy explains their global compatibility.

14. How does deppy compare to classic dataflow equations?

Traditional approach. Classic dataflow analysis solves mutually recursive equations over program points until a fixed point is reached.

Deppy analogue. Deppy retains that local propagation picture, but interprets the solved facts as local sections. The extra mathematical step is to ask which pairwise disagreements are merely artifacts of local assignments and which survive quotienting as genuine obstructions.

Relationship. Relationship: direct extension. Dataflow equations give the local algebra; deppy adds quotient structure and obstruction classification.

15. How does deppy compare to constraint-based inference?

Traditional approach. Constraint-based systems collect equations or implications and solve them globally to infer types, effects, or permissions.

Deppy analogue. Deppy can be read as organizing many local constraints into a cover, then checking whether the constraint solutions agree on overlaps. The point is not only to solve the

constraints, but to compute a structured witness when the local solutions refuse to assemble.

Relationship. Relationship: generalization in organization. Constraint solving remains present, but deppy adds a topology over where constraints arise.

16. How does deppy compare to lattice-theoretic fixed points?

Traditional approach. The classical story says analyses are monotone operators on complete lattices, and correctness follows from fixed-point theorems.

Deppy analogue. Deppy still needs fixed points for local propagation, especially in its hybrid checker and core analyses. What is new is that the fixed point is not the end of the story: one then computes how those local fixed-point facts fit together globally through Cech coboundaries and cohomology.

Relationship. Relationship: layered generalization. Fixed points remain the local engine; cohomology is the global post-processing geometry.

17. How does deppy compare to Galois connections?

Traditional approach. Galois connections justify abstraction and concretization, providing the mathematical backbone for sound abstract interpretation.

Deppy analogue. Deppy can sit on top of abstract domains justified by Galois connections, but its novel move is elsewhere. Once an abstraction yields local facts, deppy treats those facts as sections and studies gluing obstructions among them.

Relationship. Relationship: orthogonal complement. Galois connections explain abstraction quality; deppy explains global compatibility among abstract facts.

18. How does deppy compare to compositional semantics?

Traditional approach. Compositional semantics insists that the meaning of a whole be determined by the meanings of its parts and their mode of combination.

Deppy analogue. Deppy is deeply compositional because every chapter of its mathematics is about local-to-global assembly. The extra claim is that failed composition is itself mathematically observable and classifiable through obstruction classes rather than treated as an unstructured failure.

Relationship. Relationship: strengthened compositionality. Deppy adds a calculus of failure-to-compose, not just a calculus of composition.

19. How does deppy compare to modular verification?

Traditional approach. Modular verification isolates components behind interfaces so that proofs scale and local changes do not invalidate everything.

Deppy analogue. Deppy's Mayer-Vietoris story is the mathematically precise version of that intuition. Subprograms contribute their own local obstruction data, and boundary interactions appear as connecting terms rather than forcing full recomputation from scratch.

Relationship. Relationship: exact compositional lift. The modular intuition is retained, but with an exact sequence rather than only engineering discipline.

20. How does deppy compare to the usual soundness-versus-completeness tradeoff?

Traditional approach. Traditional tools usually sacrifice completeness for tractability, or gain completeness only inside a sharply delimited fragment.

Deppy analogue. Deppy is honest about the same tradeoff, but relocates one part of it. Its equivalence criterion is claimed complete relative to the chosen cover, so the approximation burden sits in cover granularity rather than in the descent principle itself.

Relationship. Relationship: conceptual reframing. The tradeoff remains, but deppy isolates where the incompleteness actually lives.

Chapter 2

Specifications, contracts, and type structure

This chapter focuses on how deppy relates to the specification languages and type disciplines that traditionally carry proof obligations.

21. How does deppy compare to design by contract?

Traditional approach. Design by contract writes explicit preconditions, postconditions, and invariants, usually checked at runtime or by a verifier.

Deppy analogue. Deppy's surface syntax keeps that spirit through assume, guarantee, and check, but the hybrid core interprets those clauses across intent, structural, semantic, proof, and code layers. A contract is no longer a single assertion site; it is part of a multi-layer coherence problem.

Relationship. Relationship: direct generalization. Design by contract appears as the public-facing syntax of a richer layered type object.

22. How does deppy compare to refinement types?

Traditional approach. Refinement types enrich plain types with predicates such as x is an int and x is nonzero, enabling SMT-backed safety checking.

Deppy analogue. Deppy inherits refinement reasoning almost literally at the structural layer. Its novelty is to embed refinements into a presheaf over sites and into a trust lattice over evidence levels, so the system can distinguish local predicate discharge from global coherence across layers.

Relationship. Relationship: conservative extension. Refinement typing is one core ingredient inside deppy's larger hybrid construction.

23. How does deppy compare to dependent types?

Traditional approach. Dependent types let types mention values directly, so specifications and implementations can be unified in one language.

Deppy analogue. Deppy's hybrid dependent types pursue the same ambition, but relax the evidence regime. Some predicates are solver decidable, some are oracle judged, some are Lean proved, and coherence across those evidential modes is itself checked.

Relationship. Relationship: pragmatic weakening and broadening. Deppy is not a pure dependent type theory, but a hybrid one aimed at mixed evidence.

24. How does deppy compare to liquid types?

Traditional approach. Liquid types occupy a sweet spot where refinements come from a decidable predicate template language solved by SMT.

Deppy analogue. Deppy agrees at the structural layer whenever the property is SMT friendly. It departs when the property is semantically rich, because then the obligation can move into the oracle or proof layers instead of being rejected outright as outside the liquid fragment.

Relationship. Relationship: strict generalization in evidence space, not in decidability. Deppy handles some liquid cases exactly and some richer cases only with lower-trust evidence.

25. How does deppy compare to gradual typing?

Traditional approach. Gradual typing mixes static and dynamic guarantees by inserting unknown types and runtime checks where static proof is absent.

Deppy analogue. Deppy mixes stronger and weaker evidence too, but with a different axis. The key variation is not between static and dynamic types, but between trust levels and predicate kinds, all recorded explicitly in the oracle monad and trust lattice.

Relationship. Relationship: analogous hybridization. Both systems manage partial knowledge, but deppy hybridizes proof strength rather than only type precision.

26. How does deppy compare to effect systems?

Traditional approach. Effect systems track side effects such as state, exceptions, or IO so that programs can be checked against behavioral disciplines.

Deppy analogue. Deppy can express some effect-like obligations as local facts at call results, resource sites, or protocol sites, but it is not yet a first-class effect calculus. Its strength lies in detecting when local effect assumptions fail to glue across a program slice.

Relationship. Relationship: partial analogue. Effect summaries can be encoded as sections, but deppy is not natively an effect algebra.

27. How does deppy compare to session types?

Traditional approach. Session types specify communication protocols as structured type-level conversations, proving that endpoints follow dual protocols.

Deppy analogue. Deppy has a family resemblance because protocol conformance can be viewed as local compatibility plus global gluing, especially across call and message boundaries. Still, it

lacks the dedicated duality machinery and channel discipline of a real session-type system.

Relationship. Relationship: suggestive but partial analogue. Deppy can host protocol obligations, yet session types remain more specialized and stronger for communication structure.

28. How does deppy compare to typestate?

Traditional approach. Typestate tracks how the legal operations on an object depend on its current state, such as open versus closed for a file handle.

Deppy analogue. Deppy is naturally comfortable with typestate-like reasoning because sites can require lifecycle predicates and report use-after-close or protocol violations. The geometric gain is that multiple local state assumptions can be studied as a gluing problem rather than only as an automaton reachability problem.

Relationship. Relationship: near-direct analogue with a richer global summary. Typestate properties become local sections and obstruction classes.

29. How does deppy compare to linear and affine types?

Traditional approach. Linear and affine systems constrain how many times a resource may be used, preventing duplication or leakage by typing discipline.

Deppy analogue. Deppy can notice downstream consequences of resource misuse, such as leaks or use-after-close, but it does not enforce linear consumption syntactically. It is therefore weaker as a prevention mechanism but broader as a post hoc semantic checker on ordinary Python.

Relationship. Relationship: orthogonal tradeoff. Linear types prevent misuse by construction; deppy detects semantic evidence of misuse in untyped code.

30. How does deppy compare to ownership and borrow checking?

Traditional approach. Ownership systems such as Rust's encode aliasing and lifetime invariants in the type system so that many memory and concurrency bugs become unrepresentable.

Deppy analogue. Deppy does not impose that discipline on source code. Instead it can infer local lifetime facts and detect mismatches, but only after the fact and with the ambient imprecision of analysis. The reward is applicability to ordinary Python rather than only to an ownership-oriented language.

Relationship. Relationship: weaker preventive guarantee but broader applicability. Borrow checking is construction-time discipline; deppy is retrospective gluing analysis.

31. How does deppy compare to algebraic data types?

Traditional approach. Algebraic data types make program structure explicit and enable case-based reasoning with strong exhaustiveness guarantees.

Deppy analogue. Deppy benefits from such structure when it exists, but its core mathematics does not depend on ADTs. Its public surface is designed for Python, so it often reconstructs semantic

cases from control flow and guards rather than from a statically declared sum type.

Relationship. Relationship: compatible substrate, not analogue. ADTs simplify the local facts that deppy would otherwise have to infer.

32. How does deppy compare to parametric polymorphism?

Traditional approach. Parametric polymorphism proves uniform behavior across all instantiations of a type variable, yielding the classic free theorems.

Deppy analogue. Deppy's stated analogue is an etale-style perspective on polymorphism: uniformity becomes the vanishing of a type-specialization obstruction. That is a geometrically ambitious reframing, though today it is more an organizing principle than a replacement for mature polymorphic proof techniques.

Relationship. Relationship: conceptual generalization with partial implementation. The analogy is wise, but not yet as complete as classic parametricity theory.

33. How does deppy compare to type classes and ad hoc polymorphism?

Traditional approach. Type classes resolve behavior by explicit dictionaries or instance search, letting programs dispatch on structured interfaces.

Deppy analogue. Deppy is less about resolving ad hoc behavior and more about checking whether the behavior observed at each specialization remains coherent. If code branches on type-specific operations, deppy can treat that as evidence against uniform gluing rather than as a typing feature to elaborate.

Relationship. Relationship: mostly orthogonal. Type classes explain dispatch; deppy evaluates the semantic consequences of the dispatched behavior.

34. How does deppy compare to subtyping?

Traditional approach. Subtyping orders types by substitutability and lets one local proof discharge another when stronger guarantees imply weaker ones.

Deppy analogue. Deppy uses that ordering directly inside its refinement lattice. The new step is that subtype-compatible local assignments are only the zero-th layer of analysis; one still asks whether all such local subtype facts match across sites and evidence layers.

Relationship. Relationship: strict inclusion. Subtyping is built in as one algebraic relation inside the broader presheaf.

35. How does deppy compare to intersection and union types?

Traditional approach. Intersection types express values satisfying multiple views at once, while union types express branching possibilities.

Deppy analogue. Deppy can host the same local logical structure in its section predicates. What is distinctive is how unions and intersections behave across overlaps: a local union may be globally inconsistent with a downstream requirement, and that inconsistency is what the cohomological quotient records cleanly.

Relationship. Relationship: local compatibility lifted to global geometry. The connectives remain familiar, but the failure mode is classified more sharply.

36. How does deppy compare to predicate transformers?

Traditional approach. Predicate transformers give a systematic semantics for how commands move assertions backward or forward.

Deppy analogue. Deppy's restriction maps are the closest analogue, but they sit inside a network of overlaps rather than inside a single linear transformer pipeline. The emphasis is not only transport of predicates, but whether the transported family becomes a cocycle or a coboundary.

Relationship. Relationship: direct analogue plus obstruction layer. Predicate transport is present, yet no longer the entire semantic story.

37. How does deppy compare to contract inference?

Traditional approach. Contract inference tries to synthesize likely preconditions, postconditions, or invariants automatically from code or usage.

Deppy analogue. Deppy can synthesize obligations from error sites and can generate surface annotations, but it also records the trust status of inferred facts. An inferred contract is therefore not merely guessed; it is attached to provenance, confidence, and potentially stronger downstream proof.

Relationship. Relationship: informed extension. Inference is combined with explicit evidence tracking rather than treated as a monolithic guess.

38. How does deppy compare to runtime assertion checking?

Traditional approach. Runtime assertion checking validates contracts on actual executions and is powerful for catching bugs that static approximations miss, but it only covers tested paths.

Deppy analogue. Deppy treats runtime evidence as one trust level rather than as the whole truth. A runtime check can contribute to the layered object, but can later be joined by solver proof, Lean proof, or semantic oracle judgment.

Relationship. Relationship: strict generalization of evidence management. Runtime checks become one point in a trust lattice rather than a separate methodology.

39. How does deppy compare to proof-carrying code?

Traditional approach. Proof-carrying code packages executable code with a proof certificate that the consumer can check independently.

Deppy analogue. Deppy approaches the same dream more flexibly. Its proof layer can carry machine-checkable evidence, but the neighboring layers can still retain weaker forms of trust when full proof is unavailable, making the artifact graded rather than all-or-nothing.

Relationship. Relationship: graded generalization. Proof-carrying code corresponds to the top end of deppy's evidence tower.

40. How does deppy compare to API contracts?

Traditional approach. API contracts document and sometimes verify obligations at module boundaries so that callers and callees can compose reliably.

Deppy analogue. Boundary sites are exactly the sort of places deppy elevates into first-class semantics. The difference is that deppy studies not only whether a given boundary contract holds locally, but how boundary facts from many sites compose and where the composition fails.

Relationship. Relationship: direct analogue with stronger composition accounting. API contracts become cover elements in the site category.

41. How does deppy compare to the classic slogan that specifications should be separate from code?

Traditional approach. Traditional methods often prefer a clean separation between program text and logical specification language for clarity and proof discipline.

Deppy analogue. Deppy deliberately blurs that line in its surface syntax while re-separating concerns internally by layer. Intent, structural predicates, semantic judgments, proofs, and code coexist in one artifact but remain distinguished in the layered presheaf.

Relationship. Relationship: managed integration. Separation is preserved semantically rather than physically.

Chapter 3

Abstract interpretation and static analysis

This chapter compares deppy to the mainstream static-analysis toolkit that propagates approximations over program structure.

42. How does deppy compare to abstract interpretation itself?

Traditional approach. Abstract interpretation computes sound approximations of program behavior by interpreting code over abstract domains with widening and narrowing as needed.

Deppy analogue. Deppy can be read as building on that exact local machinery. Its sharp departure is that it does not stop once an abstract state is found at each site; it computes a geometric summary of how those local abstract states agree, disagree, and depend on one another.

Relationship. Relationship: higher-level organizer. Abstract interpretation supplies local facts; deppy classifies their global compatibility.

43. How does deppy compare to interval analysis?

Traditional approach. Interval analysis tracks lower and upper numeric bounds, making it cheap and scalable but non-relational.

Deppy analogue. Deppy can consume interval-like facts as local sections, especially around range and division checks. The extra structure is that a missed bound is not only a local imprecision; it can become a globally localized obstruction linked to a specific overlap between upstream availability and downstream requirement.

Relationship. Relationship: conservative instance. Interval facts fit naturally inside deppy's framework, but deppy is not limited to them.

44. How does deppy compare to octagon analysis?

Traditional approach. Octagons capture constraints of the form plus or minus x plus or minus y less than c, gaining relational power while staying tractable.

Deppy analogue. Deppy has no native commitment to one numeric domain, so octagon constraints can appear as one style of structural section. What deppy adds is a cross-site quotient that explains whether relational facts from different places cohere or leave an irreducible mismatch.

Relationship. Relationship: orthogonal lift. Octagons determine local numeric precision; deppy determines global consistency of whatever local domain you choose.

45. How does deppy compare to polyhedral analysis?

Traditional approach. Polyhedral domains keep arbitrary linear inequalities, giving strong relational precision at significant computational cost.

Deppy analogue. Deppy can, in principle, host polyhedral facts wherever the solver and implementation can support them. Its asymptotic story is different: the claimed polynomial advantage concerns minimum-fix counting after local facts are gathered, not the cost of discovering those facts in the first place.

Relationship. Relationship: compatible but not a replacement. Deppy does not beat polyhedra at local numeric expressiveness; it adds a different global summary once local constraints exist.

46. How does deppy compare to relational versus non-relational domains?

Traditional approach. Traditional analyses choose between cheap non-relational domains and richer relational ones depending on precision needs.

Deppy analogue. Deppy is agnostic about that choice at the local level. Its wiser contribution is that, whichever domain you choose, it distinguishes two separate questions: local expressiveness inside a site, and global gluing across sites.

Relationship. Relationship: orthogonal decomposition. The relational choice controls section richness, while deppy controls cross-section geometry.

47. How does deppy compare to widening and narrowing?

Traditional approach. Widening accelerates convergence by sacrificing precision, and narrowing tries to recover some precision afterward.

Deppy analogue. Deppy's incremental claims are positioned precisely against this tradition. Via Mayer-Vietoris, it argues that some recomputation tasks can be decomposed exactly at the global-obstruction level instead of rerunning an approximate widening-driven fixpoint over the entire program.

Relationship. Relationship: asymptotic and exactness improvement for the global recombination step, not necessarily for all local analysis.

48. How does deppy compare to context sensitivity?

Traditional approach. Context-sensitive analyses separate different call contexts to avoid conflating facts from unrelated invocations.

Deppy analogue. Deppy can model call boundaries as separate sites and thereby benefit from context distinctions, but it does not magically remove the cost of context explosion. Its added value is that once context-indexed local facts exist, the gluing machinery can show exactly which cross-context mismatches matter.

Relationship. Relationship: compatible enhancement. Context sensitivity remains a local modeling choice inside deppy's cover.

49. How does deppy compare to flow sensitivity?

Traditional approach. Flow-sensitive analysis respects program order, unlike flow-insensitive analyses that merge facts across an entire procedure.

Deppy analogue. Deppy is naturally flow-sensitive because branch guards, call results, and error sites appear as ordered local observations. Yet the interesting gain is not just respecting order, but localizing where order-induced facts stop composing into a globally consistent story.

Relationship. Relationship: direct alignment with added localization. Flow sensitivity becomes part of the site geometry.

50. How does deppy compare to path sensitivity?

Traditional approach. Path-sensitive analyses distinguish different control-flow branches so that mutually exclusive assumptions are not merged too early.

Deppy analogue. Deppy's equivalence engine and bug detector both thrive on path distinctions, since guarded paths become local sections or aligned comparison sites. The cohomological layer then says which path-level disagreements collapse away and which persist as independent bug or inequivalence classes.

Relationship. Relationship: strong analogue with sharper quotienting. Path distinctions are preserved, then organized into obstruction space.

51. How does deppy compare to alias analysis?

Traditional approach. Alias analysis asks whether two expressions may refer to the same location, a question central to optimization and memory safety.

Deppy analogue. Deppy is not primarily an alias-calculus, but alias facts can appear as local predicates affecting resource, mutation, and protocol obligations. Its special contribution would be to treat inconsistent alias assumptions across sites as gluing problems rather than only as may or must facts.

Relationship. Relationship: partial analogue. Alias analysis remains a primitive fact source more than a replaced discipline.

52. How does deppy compare to points-to analysis?

Traditional approach. Points-to analysis approximates the heap locations reachable from references, usually trading precision for scale.

Deppy analogue. Deppy can consume points-to summaries when reasoning about nullability, effects, or resource lifetime, but it does not currently present itself as a standalone points-to engine. The analogue lives in how those heap summaries interact with downstream obligations at overlaps.

Relationship. Relationship: mostly orthogonal. Points-to information is input to deppy rather than the mathematical novelty itself.

53. How does deppy compare to shape analysis?

Traditional approach. Shape analysis reasons about linked structures and heap topology, often with rich separation or graph abstractions.

Deppy analogue. Deppy has the right local-to-global vocabulary to talk about shape facts, but not the mature dedicated abstractions of a specialized shape analyzer. Where it would shine is in organizing many local heap-shape obligations and classifying which inconsistencies are independent.

Relationship. Relationship: conceptual compatibility, current under-specialization. The geometry is promising, but the local domain is not yet as rich as classical shape analysis.

54. How does deppy compare to constant propagation?

Traditional approach. Constant propagation is the archetypal forward dataflow analysis, pushing exact constant facts where possible and top elsewhere.

Deppy analogue. Deppy can obviously host constant facts, but that is almost too easy an example. The more interesting contrast is that deppy would not stop at saying a value is or is not constant; it would ask whether the whole family of local constant assumptions is globally compatible with all required uses.

Relationship. Relationship: tiny instance inside a larger framework. Constant propagation is one local fact language deppy can organize.

55. How does deppy compare to nullness analysis?

Traditional approach. Nullness analysis tracks whether references may be null and is one of the classic practical static analyses.

Deppy analogue. Nullness is one of deppy's flagship examples. The analysis treats a dereference requirement as a local non-null section and checks whether upstream facts admit null on the overlap, with a resulting obstruction if the mismatch is satisfiable.

Relationship. Relationship: direct instance. Deppy subsumes nullness analysis as a particular family of gluing obstructions.

56. How does deppy compare to range analysis?

Traditional approach. Range analysis tracks numeric bounds to prevent errors such as overflow or out-of-bounds access.

Deppy analogue. Deppy uses exactly those local range obligations at error sites such as indexing. The key difference is that it computes a structured explanation of why the available and required ranges fail to match, and can aggregate multiple such failures into a minimum independent fix count.

Relationship. Relationship: direct instance plus global fix accounting. The local range logic is standard; the global quotient is the new part.

57. How does deppy compare to taint analysis?

Traditional approach. Taint analysis follows untrusted data into sensitive sinks, checking whether sanitization breaks the flow along the way.

Deppy analogue. Deppy treats the same problem as a disagreement between the local section allowed at the sink and the section actually transported from sources. Sanitizers act like local gluing repairs, and unresolved flows become obstructions in a security-flavored presheaf.

Relationship. Relationship: near-direct analogue with a more algebraic error model. Taint gaps are gluing failures.

58. How does deppy compare to security type systems?

Traditional approach. Security type systems prevent illicit flows by assigning confidentiality or integrity labels and enforcing typing constraints statically.

Deppy analogue. Deppy is weaker as a prevention discipline but broader in the kinds of evidence it can combine. A flow policy can be expressed as local labeled constraints, yet the interesting deppy move is to detect and localize where those labeled assumptions stop composing, even in ordinary unannotated code.

Relationship. Relationship: partial analogue. Security types are stronger when available; deppy is more retrofit friendly.

59. How does deppy compare to exception analysis?

Traditional approach. Exception analysis tracks which exceptions may be raised and where handlers may discharge them.

Deppy analogue. Deppy has an appealing interpretation here: a handler is a local repair site, and an uncaught raise is a failure to glue the available execution facts to the required safe continuation facts. The bug report becomes an obstruction localized to the escaping path.

Relationship. Relationship: direct analogue. Exception escape is another family of gluing failure.

60. How does deppy compare to interprocedural summaries?

Traditional approach. Interprocedural analysis summarizes callees so callers need not inline them completely, balancing scalability and precision.

Deppy analogue. Deppy also needs summaries at call boundaries, but it treats those summaries as sections living at call-result and argument-boundary sites. The real gain is that interactions

among summaries can be studied compositionally through cover overlaps instead of only through summary substitution.

Relationship. Relationship: compositional lift. Summaries are first-class local sections in the site category.

61. How does deppy compare to incremental static analysis?

Traditional approach. Incremental analyzers try to reuse prior results after edits rather than rerun the full analysis, typically with dependency graphs and careful invalidation.

Deppy analogue. Deppy's bold claim is that Mayer-Vietoris gives an exact mathematical decomposition for part of this problem. Instead of only engineering reuse, it frames edited and unedited regions as overlapping pieces whose obstruction data can be recombined through an exact sequence.

Relationship. Relationship: asymptotic and conceptual improvement at the recombination layer. It aims to turn a heuristic reuse problem into an exact algebraic one.

Chapter 4

Symbolic methods, solvers, and model checking

These comparisons place deppy next to the solver-aided and state-exploration traditions that dominate modern verification practice.

62. How does deppy compare to SAT solving?

Traditional approach. SAT solving asks whether propositional constraints are satisfiable and underlies many verification reductions.

Deppy analogue. Deppy uses SAT-style reasoning only as a component, for example when structural obligations reduce to boolean consistency. Its distinctive contribution is not a new satisfiability engine, but a way to organize many local SAT-like obligations and quotient their disagreements globally.

Relationship. Relationship: substrate, not peer theory. SAT is an engine inside deppy rather than the analogue of its mathematics.

63. How does deppy compare to SMT solving?

Traditional approach. SMT extends SAT with theories such as arithmetic, arrays, and algebraic datatypes, making it the workhorse of modern automated proof.

Deppy analogue. SMT is central to deppy at the structural layer: many gap predicates are discharged exactly by Z3. But deppy is wiser than equating itself with SMT, because it explicitly separates what SMT can prove from what must be tracked semantically or proved elsewhere.

Relationship. Relationship: conservative extension. SMT is the structural decision procedure inside a broader evidence stack.

64. How does deppy compare to automated theorem proving?

Traditional approach. Automated theorem provers search for proofs in logical calculi, often with specialized heuristics and complete procedures for fragments.

Deppy analogue. Deppy can consume theorem-prover output as high-trust evidence, but it does not aspire to be a generic prover. Its contribution is to place proofs, solver results, runtime checks, and oracle judgments in one common trust lattice and then ask whether they cohere.

Relationship. Relationship: evidence integration rather than replacement. Deppy is a manager of heterogeneous proof sources.

65. How does deppy compare to symbolic execution?

Traditional approach. Symbolic execution explores program paths with symbolic inputs and path conditions, querying solvers when branches or assertions matter.

Deppy analogue. Deppy's equivalence engine is especially close to symbolic execution, since it encodes guarded paths and checks whether paired path conditions admit counterexamples. The additional geometric step is to treat those path-local comparisons as a Cech complex rather than only as a bag of path obligations.

Relationship. Relationship: direct analogue with quotienting. Symbolic paths become the local charts of deppy's comparison space.

66. How does deppy compare to concolic testing?

Traditional approach. Concolic testing mixes concrete runs with symbolic constraints to steer exploration toward new behaviors.

Deppy analogue. Deppy can treat runtime evidence as one layer of trust and use solver witnesses to concretize abstract mismatches, but it is not fundamentally a search heuristic over executions. Its center of gravity is still the local-to-global organization of obligations, not input generation.

Relationship. Relationship: partial complement. Concolic testing can supply witnesses; deppy explains the structural significance of the discovered mismatch.

67. How does deppy compare to bounded model checking?

Traditional approach. Bounded model checking unrolls transitions to a finite depth and asks for counterexamples within that bound.

Deppy analogue. Deppy's path-based equivalence engine also uses bounded unrolling for loops, so there is a real methodological overlap. The difference is that deppy then interprets the resulting local comparisons through cohomology, extracting independent obstruction classes instead of stopping at the first bound witness.

Relationship. Relationship: direct instance plus richer post-processing. Bounded search produces the local data that deppy organizes globally.

68. How does deppy compare to explicit-state model checking?

Traditional approach. Explicit-state model checking enumerates reachable states or transitions and checks temporal properties over the resulting graph.

Deppy analogue. Deppy is usually coarser because it reasons over semantically chosen sites rather than the entire transition graph. That can be a strength for local-to-global safety reasoning, but it means deppy is not a drop-in replacement when full temporal branching structure is the point.

Relationship. Relationship: mostly orthogonal. Deppy compresses state structure into observation sites instead of enumerating it.

69. How does deppy compare to CEGAR?

Traditional approach. Counterexample-guided abstraction refinement iteratively refines an abstraction until spurious counterexamples disappear or a real bug remains.

Deppy analogue. Deppy has an analogous rhythm whenever a candidate obstruction is checked by a solver and either discharged or confirmed. But its mathematical identity is different: instead of refining a single abstract model only, it computes whether local disagreements are coboundaries or genuine classes after the current cover is fixed.

Relationship. Relationship: partial analogue. Both separate spurious from genuine issues, but deppy uses quotient structure where CEGAR uses refinement loops.

70. How does deppy compare to predicate abstraction?

Traditional approach. Predicate abstraction represents states by a finite set of boolean predicates, turning rich semantics into a finite abstract model.

Deppy analogue. Deppy can certainly use such predicates as structural sections, and the guide explicitly embraces boolean disagreement algebra over GF(2). The key difference is that deppy is less about the abstraction alone and more about the linear algebra of how abstract disagreements combine.

Relationship. Relationship: compatible abstraction source plus new algebraic summary. Predicate abstraction feeds the cohomological computation.

71. How does deppy compare to interpolation-based verification?

Traditional approach. Interpolation extracts intermediate assertions from proofs or infeasible paths, helping refine abstractions and localize reasoning.

Deppy analogue. Deppy shares the intermediate-assertion instinct because branch guards and overlap conditions are exactly the local assertions that mediate gluing. Yet it does not currently center Craig interpolation as the main refinement primitive; instead it studies whether the available intermediate assertions already form a globally consistent family.

Relationship. Relationship: partial analogue. Interpolants could be excellent local sections inside deppy, but are not the defining mathematics.

72. How does deppy compare to IC3 or PDR?

Traditional approach. IC3 and PDR build inductive frames that block bad states while converging toward an invariant proving safety.

Deppy analogue. Deppy's safety reasoning is less incremental in that proof-theoretic sense and more geometric in organization. It does not build a frontier of blocking clauses; it checks whether local requirements and transported facts glue, often with solver assistance on individual gaps.

Relationship. Relationship: mostly orthogonal. IC3 excels at transition-system safety; deppy excels at local-to-global consistency summaries.

73. How does deppy compare to CTL model checking?

Traditional approach. CTL checks branching-time temporal formulas over Kripke structures using fixpoint algorithms and path quantifiers.

Deppy analogue. Deppy does not natively speak branching-time temporal logic. Its comparison and bug frameworks reason about local semantic compatibility and equivalence relative to covers, not about universal or existential temporal nesting over all futures.

Relationship. Relationship: largely orthogonal. There is no honest direct analogue today beyond the general local-to-global flavor.

74. How does deppy compare to LTL model checking?

Traditional approach. LTL verifies linear-time temporal properties such as always eventually or until along execution traces.

Deppy analogue. Deppy can encode some safety-flavored temporal obligations if they appear as local protocol or lifecycle requirements, but it is not a general temporal-logic engine. Its mathematics focuses on gluing local facts, not on omega-regular acceptance over infinite traces.

Relationship. Relationship: partial and limited analogue. For trace properties, classical LTL remains the more native formalism.

75. How does deppy compare to Buchi automata constructions?

Traditional approach. Buchi automata turn infinite-trace temporal questions into automata-theoretic emptiness problems.

Deppy analogue. Deppy has no comparable automata-theoretic core. Where Buchi automata build acceptance structure over traces, deppy builds overlap structure over local semantic charts.

Relationship. Relationship: orthogonal. The two frameworks answer different shapes of question, even if both are mathematically structured.

76. How does deppy compare to k-induction?

Traditional approach. K-induction proves invariants by showing a base case for bounded steps and an inductive step over k-length windows.

Deppy analogue. Deppy's loop reasoning can use bounded unrolling, but it does not presently present itself as a k-induction engine. Its interest is in the compatibility of facts extracted from those bounded windows, not in deriving an inductive step theorem over all windows.

Relationship. Relationship: partial overlap. Bounded reasoning appears, but the proof principle is different.

77. How does deppy compare to invariant synthesis?

Traditional approach. Invariant synthesis searches for inductive assertions strong enough to prove safety or equivalence goals.

Deppy analogue. Deppy certainly needs local invariants, especially around loops, branches, and interfaces. The novel step is that even a good local invariant family may fail to glue globally, and deppy records that failure explicitly rather than silently treating it as another missed synthesis attempt.

Relationship. Relationship: enriched analogue. Invariants become local charts; cohomology explains residual global incompatibility.

78. How does deppy compare to counterexample generation?

Traditional approach. Many verifiers aim to produce a concrete input or trace demonstrating the violated property once proof fails.

Deppy analogue. Deppy does that too through solver witnesses, but it attaches the witness to a specific obstruction class and overlap location. The resulting explanation is not merely that a counterexample exists, but which local assumptions disagree and whether several witnesses represent one independent issue or many.

Relationship. Relationship: stronger diagnosis structure. Counterexamples remain useful, but they are indexed by cohomological context.

79. How does deppy compare to witness generation for positive proofs?

Traditional approach. Positive witness generation constructs proof artifacts, certificates, or model objects demonstrating satisfaction rather than violation.

Deppy analogue. Deppy's zero-th cohomology and trust lattice together serve as a graded witness story. A globally consistent section with high trust is a witness that the local obligations fit together, and that witness can be strengthened as better evidence arrives.

Relationship. Relationship: direct but graded analogue. Satisfaction witnesses are not binary; they carry trust level and provenance.

80. How does deppy compare to decidable fragments?

Traditional approach. Classical verification often carves out a fragment where full automation is decidable, rejecting or approximating anything richer.

Deppy analogue. Deppy respects decidable fragments at the structural layer but refuses to stop there. When a property escapes the fragment, it can be represented at weaker trust layers rather than excluded entirely, with the tradeoff made explicit instead of hidden.

Relationship. Relationship: pragmatic generalization. Decidable fragments remain valuable, but deppy adds a controlled escape hatch.

81. How does deppy compare to solver-aided programming?

Traditional approach. Solver-aided programming lets developers write partial programs and logical constraints, then asks a solver to fill in or verify the rest.

Deppy analogue. Deppy's hole and spec constructs clearly live in that neighborhood. The extra twist is that synthesized or solver-backed artifacts are not accepted as monolithic miracles; they join a layered proof object whose coherence with intent and code can itself be inspected.

Relationship. Relationship: direct analogue with richer artifact bookkeeping. Solver aid becomes one phase inside a tracked hybrid object.

Chapter 5

Memory, separation, and concurrency reasoning

This chapter treats the methods traditionally used for mutable state, resources, and concurrent interference, and asks where deppy really stands relative to them.

82. How does deppy compare to separation logic?

Traditional approach. Separation logic enriches Hoare reasoning with a separating conjunction, local heap ownership, and the frame rule, making it exceptionally good at modular reasoning about mutable memory.

Deppy analogue. Deppy does not currently replace that ownership calculus. Its analogue is more semantic than syntactic: local resource assumptions live at sites, and incompatibilities in how heap or lifecycle facts compose become gluing obstructions. That can detect some of the same bugs, but without the precise local heap algebra separation logic provides.

Relationship. Relationship: partial analogue, not full generalization. Separation logic is stronger for constructive heap reasoning; deppy is broader as a retrofit analyzer on ordinary code.

83. How does deppy compare to the frame rule?

Traditional approach. The frame rule says that if a command works on one piece of state, it continues to work when unrelated disjoint state is framed around it.

Deppy analogue. Deppy has a conceptual cousin in modular gluing: unaffected regions can be left untouched while local sections on the changed region are recombined through overlaps. Yet that is not the same as a syntactic frame theorem over heaps and disjointness assertions.

Relationship. Relationship: conceptual analogue. Mayer-Vietoris gives a global recomposition story, while the frame rule gives a local heap-preservation rule.

84. How does deppy compare to explicit heap models?

Traditional approach. Heap models expose locations, stores, and update operations so one can state precise memory properties and alias relations.

Deppy analogue. Deppy can sit atop such models, but it does not force one. Its own mathematical identity comes from treating heap facts, however represented, as local sections attached to observation sites.

Relationship. Relationship: substrate-level compatibility. Heap models are ingredients, not the competing formalism.

85. How does deppy compare to alias control disciplines?

Traditional approach. Alias control disciplines restrict or track sharing so that reasoning about mutation remains tractable.

Deppy analogue. Deppy can notice when sharing assumptions fail at use sites, but it does not enforce alias control by language design. It therefore behaves more like a semantic auditor than a static discipline of permission allocation.

Relationship. Relationship: weaker preventive power, broader applicability. Alias control prevents; deppy diagnoses.

86. How does deppy compare to resource invariants?

Traditional approach. Resource invariants summarize what must always hold about a shared resource, often under controlled access protocols.

Deppy analogue. Deppy maps naturally onto this because invariant obligations are local facts at resource-related sites. Its special value is to localize exactly which overlap between resource invariant, access path, and release discipline creates the contradiction.

Relationship. Relationship: direct analogue with better localization. Resource invariants become sections whose failure to align is explicitly classified.

87. How does deppy compare to concurrent separation logic?

Traditional approach. Concurrent separation logic extends resource-based reasoning to interference, locks, and shared protocols with ownership transfer.

Deppy analogue. Deppy can detect deadlocks and some protocol inconsistencies, but it lacks the full proof rules for ownership transfer and interference framing that make concurrent separation logic precise. Its niche is to identify where local concurrent assumptions disagree rather than to prove correctness by a compositional proof calculus.

Relationship. Relationship: partial and weaker analogue. Useful for bug finding, not yet a substitute for full concurrent proof theory.

88. How does deppy compare to rely-guarantee reasoning?

Traditional approach. Rely-guarantee assigns each thread assumptions about its environment and promises about its own interference, enabling modular concurrent proofs.

Deppy analogue. Deppy's overlap language fits rely-guarantee philosophically: a thread's local assumptions must coexist with the guarantees of other threads on shared boundaries. However, deppy does not yet expose rely and guarantee as first-class proof operators.

Relationship. Relationship: conceptual analogue. The mathematics of compatibility is similar, but the dedicated concurrency proof rules are not yet there.

89. How does deppy compare to Owicki-Gries reasoning?

Traditional approach. Owicki-Gries proves concurrent programs by composing thread-local assertions and then checking noninterference conditions.

Deppy analogue. That noninterference check is exactly the kind of thing deppy wants to organize geometrically: many local assertions, plus a question of whether they agree on overlaps. Still, the current implementation is closer to bug detection than to a full Owicki-Gries proof environment.

Relationship. Relationship: partial analogue. Deppy captures the shape of noninterference checking without reproducing the classical proof system wholesale.

90. How does deppy compare to linearizability proofs?

Traditional approach. Linearizability proves that concurrent objects behave as if each operation took effect atomically at some linearization point.

Deppy analogue. Deppy does not currently provide the history-based refinement machinery needed for serious linearizability proofs. At best, it can identify local protocol mismatches or race-like obstructions around candidate atomic regions.

Relationship. Relationship: mostly orthogonal today. Linearizability remains the proper formalism for that problem.

91. How does deppy compare to atomicity reasoning?

Traditional approach. Atomicity methods identify whether code blocks can be viewed as indivisible with respect to interference or scheduling.

Deppy analogue. Deppy can express local assumptions that certain interleavings must not break an invariant, and then report contradictions when those assumptions fail to compose. But it does not have a native atomicity type system or serializability calculus.

Relationship. Relationship: partial analogue. Useful for spotting symptoms, not for delivering the strongest atomicity theorems.

92. How does deppy compare to happens-before analysis?

Traditional approach. Happens-before analysis infers partial orders among events to reason about races and synchronization.

Deppy analogue. Deppy can use ordering facts as local predicates, especially in concurrency bug patterns such as check-then-act or lock ordering. The geometric value is that one can ask whether those local order assumptions fit together globally across overlapping code regions.

Relationship. Relationship: compatible layer above an ordering analysis. Happens-before facts are local inputs to deppy's gluing checks.

93. How does deppy compare to data-race freedom analyses?

Traditional approach. Race-freedom tools prove that unsynchronized conflicting accesses cannot occur, often by ownership, lockset, or happens-before reasoning.

Deppy analogue. Deppy has concrete race-pattern detection, but it is not a complete race-freedom calculus. It identifies where the local assumptions required for a safe access do not glue with the actual synchronization story seen upstream.

Relationship. Relationship: bug-oriented partial analogue. Deppy detects obstruction-shaped race risks more readily than it proves global race freedom.

94. How does deppy compare to deadlock detection?

Traditional approach. Deadlock analyses study waits-for graphs, lock ordering, or protocol cycles to find states where progress halts.

Deppy analogue. Deppy directly presents deadlock as a lock-ordering presheaf with no consistent global section. That is one of its clearest wise analogies: the local lock orders are charts, and a cycle is literally a gluing obstruction rather than just a graph-theoretic symptom.

Relationship. Relationship: strong semantic recasting. The classical cycle criterion survives, but deppy gives it a cohomological interpretation.

95. How does deppy compare to lock-order graphs?

Traditional approach. Lock-order graph methods record which locks are acquired before others and flag cycles as potential deadlocks.

Deppy analogue. Deppy subsumes that specific technique elegantly. The graph edges become local transition or ordering facts, and the absence of a consistent global ordering section corresponds to the classical cycle while also fitting the general obstruction vocabulary used elsewhere.

Relationship. Relationship: direct instance with conceptual unification. Lock-order graphs become one species of presheaf inconsistency.

96. How does deppy compare to actor or message-passing verification?

Traditional approach. Message-passing verification reasons about mailbox protocols, ordering, and absence of stuck states in distributed interactions.

Deppy analogue. Deppy can phrase message protocol facts as local sections at send, receive, and call boundaries, but it lacks the mature event-structure semantics of actor-focused tools. Its comparative advantage would lie in unifying protocol mismatches with the same obstruction machinery used for ordinary API misuse.

Relationship. Relationship: partial analogue. Promising at the interface layer, incomplete as a full distributed semantics.

97. How does deppy compare to session-typed concurrency specifically?

Traditional approach. Session-typed concurrency proves protocol duality and deadlock-freedom through a typed discipline tightly coupled to channel structure.

Deppy analogue. Deppy can detect protocol inconsistencies after the fact, but it does not synthesize or check the full duality discipline. Its best analogue is to treat a protocol mismatch as a failure of local protocol sections to glue across the communication boundary.

Relationship. Relationship: weaker semantic checker versus stronger type discipline. Good for retrofit analysis, not a substitute for session typing.

98. How does deppy compare to memory-model reasoning?

Traditional approach. Memory-model reasoning distinguishes sequential consistency from weaker models and proves which reorderings are allowed.

Deppy analogue. Deppy currently lives far above that level. It can use synchronization and ordering facts, but it does not offer a native logic for weak-memory reorderings or fence placement.

Relationship. Relationship: largely orthogonal. Weak-memory verification remains a specialized topic outside deppy's current sweet spot.

99. How does deppy compare to ownership transfer protocols?

Traditional approach. Ownership-transfer protocols formalize when rights over a resource move between components or threads.

Deppy analogue. Deppy's trust and protocol machinery can record that a resource is expected to be open, closed, held, or released at different sites, so transfer facts can be modeled. Yet the current implementation reasons more by local obligation mismatches than by a full transfer calculus.

Relationship. Relationship: partial analogue. The state changes can be represented, but the dedicated algebra is thinner than in ownership logics.

100. How does deppy compare to region types?

Traditional approach. Region systems discipline allocation and deallocation by associating values with statically scoped memory regions.

Deppy analogue. Deppy can observe region-like lifetime mismatches if they manifest as use-after-close or leak-style contradictions, but it does not impose region structure in the type system of the source language.

Relationship. Relationship: weaker but broader. Region types prevent errors structurally; deppy detects their semantic shadows in conventional programs.

101. How does deppy compare to garbage-collection safety reasoning?

Traditional approach. Garbage-collection safety usually proves that memory reclamation preserves reachability and does not free live objects.

Deppy analogue. Deppy is not a GC meta-theory. It can talk about resource lifetime inconsistencies at the source-program level, but collector correctness is better served by operational, separation, or machine-checked implementation proofs.

Relationship. Relationship: orthogonal. GC safety is a lower-level proof problem than deppy's usual program-site analysis.

Chapter 6

Equivalence, refinement, and program transformation

This chapter covers the tradition of proving that two artifacts mean the same thing, or that one safely refines another.

102. How does deppy compare to plain program equivalence?

Traditional approach. Traditional equivalence asks whether two programs compute the same observable behavior for all relevant inputs and contexts.

Deppy analogue. Deppy's main theorem for this space is that equivalence corresponds to vanishing first cohomology of an isomorphism presheaf, relative to the chosen cover. Local site agreements are not enough by themselves; they must glue into a global section.

Relationship. Relationship: full semantic recasting with a claimed completeness result relative to the cover.

103. How does deppy compare to observational equivalence?

Traditional approach. Observational equivalence identifies programs that no permitted observer can distinguish by any context.

Deppy analogue. Deppy approximates this through the chosen site cover and the aligned observations it treats as semantically relevant. That is powerful but also honest: the strength of the equivalence result depends on whether the cover captures the observations that matter.

Relationship. Relationship: partial analogue with explicit observational boundary. The cover plays the role of the observer class.

104. How does deppy compare to contextual equivalence?

Traditional approach. Contextual equivalence quantifies over all surrounding contexts, which is semantically robust but often hard to prove directly.

Deppy analogue. Deppy does not quantify over arbitrary syntactic contexts in the traditional way. Instead it builds a common refinement of the two programs' observation sites and asks whether the local isomorphisms on that refinement glue globally.

Relationship. Relationship: relative analogue. Deppy trades universal contexts for a cover-relative descent criterion.

105. How does deppy compare to bisimulation?

Traditional approach. Bisimulation proves equivalence by matching transitions step for step, often giving elegant coinductive proofs for processes or state machines.

Deppy analogue. Deppy is not stepwise in that sense. Its local comparisons may resemble matched states, but the global criterion is about gluing those local matches rather than coinductively sustaining them forever across a transition relation.

Relationship. Relationship: conceptual cousin, not direct replacement. Both prove sameness via local correspondences, but the proof principle differs.

106. How does deppy compare to simulation relations?

Traditional approach. Simulation relations show that one system mimics another, often establishing refinement or one-way correctness properties.

Deppy analogue. Deppy can encode directional local compatibility if one side's behaviors are required to fit inside the other's, but its flagship theorem is symmetric equivalence through an isomorphism presheaf. A one-way simulation story is possible, yet less central.

Relationship. Relationship: partial analogue. Simulation is a directional specialization of the more symmetric local-gluing viewpoint.

107. How does deppy compare to refinement checking?

Traditional approach. Refinement checking proves that an implementation satisfies a more abstract specification by preserving allowed behaviors and rejecting forbidden ones.

Deppy analogue. Deppy fits refinement naturally because each site can compare implementation facts to specification facts, and global success means those local refinements glue. The attractive part is that refinement failures can be localized as obstruction classes rather than returned as a single coarse verdict.

Relationship. Relationship: strong analogue. Refinement becomes a local-to-global compatibility problem with explicit localization.

108. How does deppy compare to trace inclusion?

Traditional approach. Trace inclusion checks whether all traces of one system are also traces of another, often for safety-oriented refinement.

Deppy analogue. Deppy is usually not trace-complete in that sense because its cover compresses behavior into observation sites. It can approximate trace-sensitive questions where the chosen sites capture the relevant control structure, but it does not natively compute language inclusion over all traces.

Relationship. Relationship: partial and weaker analogue. Trace inclusion remains the more exact formalism when traces themselves are the semantic object.

109. How does deppy compare to noninterference proofs?

Traditional approach. Noninterference proves that changes in high-security inputs cannot affect low-security observations.

Deppy analogue. Deppy can phrase noninterference as a relational equivalence or compatibility problem between paired observations that differ only in secret inputs. Its obstruction language is attractive here because a leak appears as a local mismatch that refuses to glue away.

Relationship. Relationship: strong analogue for finite observational setups. The geometric framing is especially natural for leak localization.

110. How does deppy compare to compiler correctness?

Traditional approach. Compiler correctness proves that source and target programs preserve semantics across translation, often by simulation, logical relations, or refinement.

Deppy analogue. Deppy can compare translated artifacts site by site and certify equivalence relative to an aligned cover, but it does not currently provide the deep language-to-language metatheory of a CompCert-style proof. Its strength is more in translation validation than in proving a compiler once and for all.

Relationship. Relationship: partial analogue. Excellent for individual translation comparisons, weaker for foundational compiler-theorem development.

111. How does deppy compare to translation validation?

Traditional approach. Translation validation checks each compiled artifact after the fact rather than proving the compiler correct for all inputs in advance.

Deppy analogue. This is one of deppy's most natural homes. The isomorphism presheaf can compare a source routine and its transformed version directly, and a nontrivial obstruction class pinpoints which local rewrite stopped preserving behavior.

Relationship. Relationship: strong direct analogue. Translation validation is a practical face of deppy's descent-based equivalence story.

112. How does deppy compare to superoptimization?

Traditional approach. Superoptimization searches for a cheaper program that is provably equivalent to the original, often using solver-backed equivalence checking in the loop.

Deppy analogue. Deppy can serve as the equivalence-checking side of that workflow and can even classify which candidate mismatches are independent. But it is not itself a search procedure over the optimization space.

Relationship. Relationship: complementary component. Deppy is the verifier and explainer, not the optimizer search engine.

113. How does deppy compare to peephole equivalence checking?

Traditional approach. Peephole equivalence checking validates small local rewrites, typically over instruction sequences or tiny expressions.

Deppy analogue. Deppy's local-site machinery is overkill in the tiny case but still fits well. The advantage appears when many local rewrites interact, because independent local agreements and disagreements can be assembled into a global equivalence diagnosis rather than checked in isolation.

Relationship. Relationship: generalization from local rewrite proofs to globally interacting local proofs.

114. How does deppy compare to algebraic rewriting?

Traditional approach. Algebraic rewriting uses equalities such as commutativity or associativity to transform terms while preserving meaning.

Deppy analogue. Deppy explicitly frames some implementation symmetries in a Galois-cohomology style, which is a wise way to say that some structural differences should not count as substantive inequivalences. It separates symmetry-explained variation from residual unexplained obstruction.

Relationship. Relationship: conceptual extension. Rewriting laws become symmetry generators inside a broader equivalence classifier.

115. How does deppy compare to partial evaluation correctness?

Traditional approach. Partial evaluation specializes code with known inputs, and correctness requires that the specialized residual program preserve behavior for the remaining inputs.

Deppy analogue. Deppy can compare original and residual code along the remaining free-input interface, and local mismatches become obstruction sites. It does not specialize code by itself, but it offers a clean after-the-fact correctness checker for specialization outcomes.

Relationship. Relationship: direct validation analogue. Partial evaluation supplies the transformation; deppy supplies relational certification.

116. How does deppy compare to common subexpression elimination correctness?

Traditional approach. CSE correctness proves that sharing a reused computation does not change observable behavior, especially in the presence of effects or exceptions.

Deppy analogue. Deppy is good at exactly the place where CSE becomes subtle: side effects, mutation, or exceptional control can create local mismatches even when expressions look algebraically equal. A failed gluing witness can show that the seemingly harmless sharing changed a boundary observation.

Relationship. Relationship: practical relational checker. It is especially strong when the transformed code is ordinary source rather than proof objects.

117. How does deppy compare to dead-code elimination correctness?

Traditional approach. Dead-code elimination removes computations believed irrelevant to observables, and correctness requires proving that the removed code had no semantic effect.

Deppy analogue. Deppy checks this by comparing pre and post transformation observation sites. If a supposedly dead branch influenced a required local section elsewhere, the isomorphism presheaf will fail to glue and the obstruction will reveal the hidden dependency.

Relationship. Relationship: direct validation analogue with good localization of hidden effects.

118. How does deppy compare to loop optimization correctness?

Traditional approach. Loop optimizations such as interchange, fusion, or strength reduction require preserving subtle dependencies and numeric behavior.

Deppy analogue. Deppy can compare optimized and unoptimized loops through bounded symbolic encodings and aligned observations, which is valuable for finding mismatches quickly. Still, for deep parametric loop proofs, dedicated dependence analysis and theorem proving may remain stronger.

Relationship. Relationship: mixed. Strong for validation-by-comparison, weaker than deep transformation-specific proof frameworks.

119. How does deppy compare to CPS transformation correctness?

Traditional approach. CPS correctness proves that converting direct style into continuation-passing style preserves semantics despite dramatic control-structure change.

Deppy analogue. Deppy can, in principle, compare the two artifacts if the cover captures the relevant observations, but this is a demanding test of any finite observational alignment. The method is more naturally suited to source-level variants than to radical representation shifts unless the cover is designed very carefully.

Relationship. Relationship: ambitious partial analogue. Possible, but not where deppy's current implementation is most obviously strongest.

120. How does deppy compare to defunctionalization correctness?

Traditional approach. Defunctionalization replaces higher-order functions with first-order tags and an apply function, requiring a semantic correspondence proof.

Deppy analogue. As with CPS, deppy can act as a translation validator when the observation cover aligns the two worlds. The geometric framework is attractive because it focuses on local observation agreement rather than on mirroring every internal representation choice.

Relationship. Relationship: partial analogue. Useful as a checker, less mature as a foundational proof method for representation-changing transformations.

121. How does deppy compare to semantics-preserving refactoring?

Traditional approach. Refactoring tools promise that structural changes such as renaming, extraction, or reordering do not change behavior.

Deppy analogue. Deppy's equivalence story is almost tailor-made for this use case. Local syntactic changes can be ignored when they correspond to known symmetries, while genuine behavioral shifts surface as nontrivial obstructions in the aligned comparison cover.

Relationship. Relationship: strong practical analogue. It turns refactoring correctness into a structured local-to-global equivalence check.

Chapter 7

Temporal reasoning, termination, and reactive properties

This chapter asks where deppy's local-to-global machinery reaches its limits when the main object of study is time, progress, or infinite behavior.

122. How does deppy compare to termination proofs?

Traditional approach. Termination proofs show that every execution eventually halts, often by well-founded measures or semantic arguments.

Deppy analogue. Deppy can represent some local ranking-like obligations, but termination is not its flagship theorem. Its native mathematics is better at compatibility of local safety facts than at establishing global well-founded descent over all executions.

Relationship. Relationship: partial and currently secondary analogue. Classical termination methods remain more direct.

123. How does deppy compare to ranking functions?

Traditional approach. Ranking functions map states into a well-founded order that decreases on every loop or recursive step, proving termination automatically for many programs.

Deppy analogue. Such functions can appear as structural or proof-layer obligations inside deppy, especially through generated invariants and Lean proofs. Yet deppy itself does not replace the ranking-function search problem; it mainly offers a place to record and integrate the result.

Relationship. Relationship: evidence-hosting analogue. Ranking functions are local proof ingredients inside deppy, not the new mathematics.

124. How does deppy compare to well-founded relations?

Traditional approach. Well-founded relations generalize ranking functions and support termination and induction arguments beyond simple numeric descent.

Deppy analogue. Deppy can certainly carry a proof that some recursion is well founded, but that proof lives in the proof layer rather than emerging from the cohomological core. The site-and-gluing story is orthogonal to the existence of a well-founded order.

Relationship. Relationship: orthogonal complement. Well-foundedness proves progress; deppy organizes multi-source evidence around the rest of the artifact.

125. How does deppy compare to liveness verification?

Traditional approach. Liveness properties say that something good eventually happens, which usually requires temporal reasoning over possibly infinite executions.

Deppy analogue. Deppy is much weaker here than for safety. A local mismatch framework naturally captures bad finite overlaps and protocol inconsistencies, but eventuality over all futures is not yet a native object of its current engines.

Relationship. Relationship: mostly orthogonal. Deppy is safety-first and localization-first, not full liveness logic.

126. How does deppy compare to safety proofs?

Traditional approach. Safety says that nothing bad ever happens, and many static analyzers are engineered primarily around this shape of property.

Deppy analogue. This is deppy's natural habitat. Error sites define local formulations of badness, and the analysis asks whether upstream facts can be glued to satisfy those local safety requirements everywhere.

Relationship. Relationship: strong direct analogue. Deppy is a structured safety-analysis framework with unusually explicit global obstruction summaries.

127. How does deppy compare to fairness assumptions?

Traditional approach. Fairness assumptions control which infinite schedules are considered admissible so liveness claims do not fail for pathological starvation traces.

Deppy analogue. Deppy has no direct fairness logic. If fairness enters, it would have to appear as an imported semantic assumption or theorem rather than as a first-class operator in the core engine.

Relationship. Relationship: orthogonal. Fairness belongs to temporal and concurrency metatheory more than to deppy's current local-observation framework.

128. How does deppy compare to temporal logic generally?

Traditional approach. Temporal logics such as LTL and CTL add operators for next, until, always, and eventually so one can state sequence-level properties directly.

Deppy analogue. Deppy instead talks about covers, restrictions, and gluing. That is powerful for structural consistency questions, but it does not supply the same native syntax or semantics for

long-range temporal modalities.

Relationship. Relationship: different axis. Temporal logic is about time-indexed modalities; deppy is about locality-indexed compatibility.

129. How does deppy compare to reactive synthesis?

Traditional approach. Reactive synthesis automatically builds a system from a temporal specification that must hold against all environment behaviors.

Deppy analogue. Deppy's hole and spec mechanisms suggest a synthesis story, but the implemented mathematics is more comfortable validating or diagnosing candidate artifacts than solving full adversarial synthesis games. Its best role today is as a verifier inside a synthesis loop rather than the game solver itself.

Relationship. Relationship: complementary tool, not replacement. Strong on checking, weaker on strategy construction.

130. How does deppy compare to monitor synthesis?

Traditional approach. Monitor synthesis turns specifications into runtime monitors that watch executions for violations.

Deppy analogue. Deppy can interpret runtime checks as one trust level and can compile some local obligations into executable checks, but that is not the same as deriving optimal monitors from temporal formulas. Its runtime aspect is subordinate to a broader evidence story.

Relationship. Relationship: partial analogue. Useful for runtime evidence, but not a dedicated monitoring formalism.

131. How does deppy compare to runtime verification?

Traditional approach. Runtime verification checks actual executions against specifications and is invaluable when static precision is limited.

Deppy analogue. Deppy explicitly incorporates runtime evidence as one trust level in its lattice. The wise difference is that runtime success neither overstates itself as full proof nor disappears after the run; it becomes a tracked piece of the layered verification object.

Relationship. Relationship: strict generalization of evidence bookkeeping. Runtime verification is one coordinate in deppy's trust space.

132. How does deppy compare to hybrid-systems verification?

Traditional approach. Hybrid-systems methods verify programs with continuous dynamics, usually by differential invariants and reachability over ODEs.

Deppy analogue. Deppy does not currently speak the language of continuous state and differential equations. Its sheaf vocabulary may sound mathematically grand, but the implemented engine is aimed at discrete software semantics, not continuous control systems.

Relationship. Relationship: orthogonal. Hybrid-systems verification is outside deppy's present scope.

133. How does deppy compare to probabilistic model checking?

Traditional approach. Probabilistic model checking reasons about likelihoods of events, expected values, and quantitative temporal properties in stochastic systems.

Deppy analogue. Deppy has no dedicated probability monad or stochastic semantics in its current core. One could imagine local probability facts as sections, but that would be a future extension rather than a present analogue.

Relationship. Relationship: currently orthogonal. Classical probabilistic tools remain the right answer here.

134. How does deppy compare to quantitative verification?

Traditional approach. Quantitative verification tracks costs, probabilities, energy, or robustness margins rather than only boolean satisfaction.

Deppy analogue. Deppy mostly computes boolean disagreement structure over GF(2) when doing its core obstruction accounting. That makes it elegant for independence counting, but less native for rich quantitative objectives unless those are separately encoded.

Relationship. Relationship: limited analogue. Quantitative reasoning is not the central geometry of the current system.

135. How does deppy compare to cost analysis?

Traditional approach. Cost analysis estimates time, space, or resource consumption symbolically, often by recurrences or abstract interpretation.

Deppy analogue. Deppy can certainly host cost obligations in contracts, but it does not currently provide the dedicated recurrence solving and asymptotic machinery of cost analyzers. Its better role is to track whether claimed cost-related facts agree across evidence layers and program boundaries.

Relationship. Relationship: orthogonal with possible integration. Cost analysis supplies facts that deppy could manage, rather than the reverse.

136. How does deppy compare to amortized analysis?

Traditional approach. Amortized analysis proves average resource bounds across sequences of operations via potentials or accounting arguments.

Deppy analogue. That style of reasoning is global across traces and data-structure histories, not merely local across code sites. Deppy could record the resulting theorem, but it does not yet provide a native calculus of potentials or resource credits.

Relationship. Relationship: mostly orthogonal. Amortized proofs remain a specialized proof discipline.

137. How does deppy compare to real-time verification?

Traditional approach. Real-time verification reasons about deadlines and clock constraints, usually with timed automata or schedulability logic.

Deppy analogue. Deppy has no first-class clock semantics. Timing facts could be imported as local predicates, but there is no native analogue of a timed-automata engine or region-graph decision procedure.

Relationship. Relationship: orthogonal. Real-time verification lies outside deppy's current core competency.

138. How does deppy compare to schedulability analysis?

Traditional approach. Schedulability analysis proves that task sets meet timing deadlines under particular scheduling policies.

Deppy analogue. Deppy does not currently model scheduler policy or timing composition at that granularity. Its concurrency reasoning is about protocol and resource mismatches rather than deadline arithmetic.

Relationship. Relationship: orthogonal. Schedulability remains a domain-specific quantitative analysis.

139. How does deppy compare to fault-tolerance proofs?

Traditional approach. Fault-tolerance proofs establish that a system preserves safety or recovery guarantees under crashes, omission faults, or Byzantine behavior.

Deppy analogue. Deppy can capture some recovery obligations as local contracts, and its trust lattice is naturally provenance aware, but it lacks the distributed adversarial semantics of serious fault-tolerance verification. Its current geometry is better suited to code-level mismatches than system-level fault models.

Relationship. Relationship: partial and high-level analogue only. Full fault models are outside today's implementation envelope.

140. How does deppy compare to recovery-protocol verification?

Traditional approach. Recovery protocols prove that after an error the system returns to a safe state or preserves specific invariants.

Deppy analogue. Deppy can express local post-error obligations and check whether guards, handlers, and resource disciplines are sufficient to glue back to a safe continuation. That gives a useful code-centric view of recovery even if it is not a full protocol calculus.

Relationship. Relationship: partial direct analogue. Good for local recovery structure, not yet for full distributed recovery proofs.

141. How does deppy compare to hyperproperties?

Traditional approach. Hyperproperties relate multiple executions at once, as in noninterference or observational determinism.

Deppy analogue. Deppy's equivalence and relational checking machinery is a promising fit because it already aligns multiple artifacts or runs and asks for local agreement. In that sense, its isomorphism presheaf is naturally hyperproperty flavored even if not packaged in the standard HyperLTL vocabulary.

Relationship. Relationship: strong conceptual analogue. Many two-run hyperproperties look like deppy-style gluing problems.

Chapter 8

Proof assistants, synthesis, and automation

Here the question is not only what can be proved, but how proofs, generated code, and machine assistance are integrated into the artifact.

142. How does deppy compare to interactive theorem proving?

Traditional approach. Interactive theorem proving uses proof assistants to let humans build machine-checked derivations of rich semantic theorems.

Deppy analogue. Deppy treats that mode of proof as the top of its trust lattice rather than as the only acceptable evidence. A Lean proof is ideal, but weaker evidence can coexist while the artifact remains usable and explicitly graded.

Relationship. Relationship: pragmatic generalization of workflow. Interactive proof remains the gold standard, but not the only recognized contributor.

143. How does deppy compare to Lean, Coq, or Isabelle style verification?

Traditional approach. These systems embed programs and proofs in expressive logics where soundness is checked by a trusted kernel.

Deppy analogue. Deppy borrows their aspiration for kernel-level certainty at the proof layer while keeping ordinary Python as the programming surface. Its compromise is to accept a spectrum of evidence and then ask whether the layers remain coherent with the eventual code artifact.

Relationship. Relationship: hybridization rather than replacement. Kernel-checked proof is one layer inside a more permissive pipeline.

144. How does deppy compare to tactic-based proof automation?

Traditional approach. Tactic systems script proof search at the meta level, helping humans discharge repetitive proof obligations in assistants.

Deppy analogue. Deppy can benefit from tactic-generated proofs when Lean obligations are produced, but tactics are not its native analysis language. They are one source of high-trust evidence consumed by the layered checker.

Relationship. Relationship: supporting tool. Tactics strengthen deppy artifacts but do not define their structure.

145. How does deppy compare to proof by reflection?

Traditional approach. Reflection proves properties by reducing them to computation inside the proof assistant, gaining efficiency and trust.

Deppy analogue. Deppy has a similar aspiration when structural checks are delegated to solvers and then potentially reified into proof artifacts, but it does not currently collapse its whole pipeline into reflective computation. The hybrid approach accepts heterogeneous evidence instead of a single reflective kernel story.

Relationship. Relationship: partial analogue. Reflection is a stronger kernel-centric ideal than deppy's mixed-evidence workflow.

146. How does deppy compare to certified extraction?

Traditional approach. Certified extraction turns proved functional artifacts into executable programs while preserving semantic guarantees.

Deppy analogue. Deppy inverts that flow in an interesting way: it starts from ordinary code and tries to attach proof, semantic judgment, and runtime evidence back onto it. Its aim is not extraction from proof terms, but certification of already existing or synthesized code artifacts.

Relationship. Relationship: dual workflow. Extraction moves from proof to code; deppy moves from code toward graded proof.

147. How does deppy compare to certified decision procedures?

Traditional approach. Certified decision procedures produce answers together with machine-checkable certificates, reducing trust in complex solvers.

Deppy analogue. Deppy would welcome such procedures because they raise evidence from the structural layer toward the proof layer. Its trust lattice is exactly the right place to record that upgrade, with provenance showing how the stronger certificate was obtained.

Relationship. Relationship: direct fit. Certified procedures are a mechanism for moving upward in deppy's evidence order.

148. How does deppy compare to proof automation generally?

Traditional approach. Proof automation tries to discharge as much as possible without manual intervention, often via solvers, rewriting, search, or machine-learning guidance.

Deppy analogue. Deppy is automation friendly but skeptical in the right way. It automates what it can, yet always records how the obligation was discharged and at what trust level, instead of flattening all automation outcomes into an undifferentiated proven status.

Relationship. Relationship: wiser evidence accounting. Automation is welcomed, but never allowed to hide its epistemic status.

149. How does deppy compare to program synthesis?

Traditional approach. Program synthesis constructs code from specifications, examples, or constraints, turning verification conditions into generative guidance.

Deppy analogue. Deppy's hole and spec constructs are synthesis oriented, but the framework is unusual because synthesized code is immediately placed back into the same layered verification object. The synthesized artifact is treated as a candidate section that still must cohere with intent, proof, and code-level behavior.

Relationship. Relationship: synthesis-plus-certification loop. The analogue is strong, but deppy emphasizes post-synthesis coherence more than raw search power.

150. How does deppy compare to syntax-guided synthesis?

Traditional approach. SyGuS constrains the search space with a grammar and asks a solver to find a term satisfying the specification.

Deppy analogue. Deppy can live comfortably on top of such synthesis because its hole descriptions and structure can delimit candidate code. Yet SyGuS solves the search problem, while deppy supplies the richer artifact that contextualizes and validates the resulting candidate.

Relationship. Relationship: complementary. SyGuS is a search formalism; deppy is a verification-and-integration shell around candidate code.

151. How does deppy compare to CEGIS?

Traditional approach. Counterexample-guided inductive synthesis alternates between guessing a candidate and refuting it with counterexamples until convergence.

Deppy analogue. Deppy can participate naturally in that loop by producing exactly the sort of localized counterexamples and incompatibility summaries a synthesizer needs. What it adds beyond raw counterexamples is a notion of independent obstruction classes, which can guide more structured repairs.

Relationship. Relationship: strong complement. Deppy is not the learner, but it is an unusually informative teacher.

152. How does deppy compare to sketching?

Traditional approach. Sketching leaves holes in a program and relies on a solver or synthesizer to fill them subject to assertions and examples.

Deppy analogue. Deppy's hole form is an explicit embrace of that workflow. The difference is that the filled hole inherits the full hybrid-evidence apparatus, so the final artifact remembers whether the fill was solver-proved, oracle-judged, runtime-tested, or Lean-certified.

Relationship. Relationship: direct analogue with stronger provenance. Sketches become typed and trust-annotated subproblems.

153. How does deppy compare to hole-driven development?

Traditional approach. Hole-driven development treats unfinished code as a typed obligation, using the surrounding context to guide completion.

Deppy analogue. This is almost exactly deppy's public story for holes, except that the surrounding context is richer than a pure type. It includes intent text, structural predicates, semantic checks, proof obligations, and downstream gluing constraints.

Relationship. Relationship: direct generalization. The hole is solved in a hybrid dependent context rather than in a plain type context.

154. How does deppy compare to abductive inference?

Traditional approach. Abductive inference invents the missing assumptions or conditions that would make a proof or verification attempt succeed.

Deppy analogue. Deppy naturally points toward abduction because a nontrivial obstruction says some local fact is missing or inconsistent. The difference is that the missing fact is not guessed blindly; the location and overlap structure of the obstruction tell you where a repair should live.

Relationship. Relationship: diagnosis-guided abduction. Cohomology does not just say some assumption is missing; it tells you how many independent assumptions are missing and where.

155. How does deppy compare to specification mining?

Traditional approach. Specification mining infers likely properties from executions, code patterns, or documentation so that verification can begin from partial knowledge.

Deppy analogue. Deppy can absorb mined properties into weaker trust layers rather than pretending they are fully proved. That is wise because mined properties are often useful but uncertain; provenance and confidence become first-class citizens instead of awkward metadata.

Relationship. Relationship: strong evidence-management analogue. Mined specs become tracked hypotheses inside the hybrid system.

156. How does deppy compare to invariant mining?

Traditional approach. Invariant mining guesses likely assertions that hold across observed executions or static patterns, often as seeds for stronger proofs.

Deppy analogue. Deppy benefits from those guesses the same way it benefits from mined specifications: as local sections with explicit trust and provenance. If the mined invariant fails to cohere

with stronger layers or downstream use sites, the mismatch is visible rather than silently ignored.

Relationship. Relationship: compatible enhancement. Deppy turns mined invariants into accountable verification artifacts.

157. How does deppy compare to test generation?

Traditional approach. Test generation tries to cover behaviors or satisfy path conditions so that faults surface empirically.

Deppy analogue. Deppy can use solver-generated witnesses and runtime checks, but testing remains only one source of evidence. The richer contribution is that generated tests can be tied back to specific obstruction classes rather than sitting as isolated failing inputs.

Relationship. Relationship: generalization of diagnosis context, not of generation. Testing supplies evidence; deppy explains its structural significance.

158. How does deppy compare to fuzzing?

Traditional approach. Fuzzing searches for crashing or inconsistent behaviors by mutating inputs, usually prioritizing coverage or novelty.

Deppy analogue. Deppy does not compete with fuzzing as a search strategy. What it can do is absorb fuzz-found failures into a structured artifact whose local mismatch, confidence, and relation to other failures are explicit.

Relationship. Relationship: complementary. Fuzzing finds candidate bugs; deppy classifies and relates them.

159. How does deppy compare to metamorphic testing?

Traditional approach. Metamorphic testing checks whether known semantic relations among multiple inputs and outputs are preserved even when no direct oracle exists.

Deppy analogue. That is philosophically close to deppy's relational style, especially in equivalence checking. The difference is that deppy tries to prove or refute relational compatibility symbolically and structurally, not only to sample it through transformed tests.

Relationship. Relationship: strong conceptual analogue. Metamorphic relations are finite empirical shadows of deppy's relational gluing claims.

160. How does deppy compare to differential testing?

Traditional approach. Differential testing runs multiple implementations on the same inputs and flags divergences as likely bugs.

Deppy analogue. Deppy's equivalence pipeline is the formalized cousin of that workflow. Rather than only observing divergence on sampled inputs, it builds local relational sections, asks for solver witnesses, and classifies independent inequivalence sources through first cohomology.

Relationship. Relationship: strict strengthening. Differential testing is empirical comparison; deppy is structured symbolic comparison with localization.

161. How does deppy compare to explainable automation?

Traditional approach. Modern verification tooling increasingly values explanations for why a proof failed, a candidate was rejected, or a warning matters.

Deppy analogue. Deppy is unusually aligned with that demand because its central objects are explanatory: local sections, overlaps, and obstruction classes naturally support site-specific narratives. It is not just searching for an answer; it is organizing why the answer has the shape it does.

Relationship. Relationship: methodological advantage. Explanation is not bolted on after the fact; it follows from the mathematics.

Chapter 9

Security, robustness, and policy reasoning

This chapter compares deppy to the formal methods used for adversarial flows, secure configuration, and trustworthy software boundaries.

162. How does deppy compare to information-flow control?

Traditional approach. Information-flow control labels data and enforces noninterference or controlled declassification through typing or semantic restrictions.

Deppy analogue. Deppy can express similar local label requirements at sources, sanitizers, and sinks, then ask whether the induced facts glue globally. Its advantage is retrofit applicability to unannotated code; its weakness is that it usually diagnoses violations rather than preventing them by language design.

Relationship. Relationship: partial analogue with broader deployment. IFC is stronger by construction; deppy is more flexible in mixed-code settings.

163. How does deppy compare to declassification reasoning?

Traditional approach. Declassification logics make explicit when certain secret-to-public flows are intentionally allowed under controlled conditions.

Deppy analogue. Deppy can interpret sanitizers, policy checks, or explicit guards as local repairs that discharge what would otherwise be an illicit flow obstruction. The important wise point is that declassification becomes a witnessed local gluing event, not an unstructured exception to policy.

Relationship. Relationship: direct analogue for policy exceptions. Allowed flows are modeled as locally justified repairs.

164. How does deppy compare to sanitization proofs?

Traditional approach. Sanitization proofs show that untrusted data is transformed enough to be safe for a sensitive sink such as SQL or HTML rendering.

Deppy analogue. Deppy handles this beautifully in its own vocabulary. The sink requires a sanitized section, the source provides a tainted section, and the sanitizer is the local morphism that may or may not convert one into the other on the overlap.

Relationship. Relationship: strong direct analogue. Sanitization is gluing repair in one of its clearest practical forms.

165. How does deppy compare to SQL injection analysis?

Traditional approach. Traditional SQL-injection analysis tracks tainted strings, structured query APIs, and sanitization barriers to keep attacker data out of query syntax.

Deppy analogue. Deppy already treats concatenated queries and missing sanitization as obstruction patterns. The wise analogue is that the sink's required local section is not just stringness but a stronger semantic predicate about query safety, and the failure to meet it is localized precisely.

Relationship. Relationship: direct instance. SQL injection is a canonical taint-flavored gluing failure in deppy.

166. How does deppy compare to XSS analysis?

Traditional approach. XSS analysis reasons about whether untrusted content can reach browser-executed contexts without appropriate escaping or policy barriers.

Deppy analogue. Deppy can phrase the same issue as a local mismatch between the semantic requirements of an HTML or script sink and the transported content provenance. Escapers and content-type checks act like local repairs; unescaped flows remain obstruction classes.

Relationship. Relationship: direct analogue. The taint story is the same, with a different sink semantics.

167. How does deppy compare to command-injection analysis?

Traditional approach. Command-injection tools track whether user-controlled text reaches shell invocation boundaries in dangerous positions.

Deppy analogue. Deppy sees that as another sink discipline problem. The call site requires a structurally safe command representation, while upstream string assembly may admit attacker influence that refuses to glue with the sink contract.

Relationship. Relationship: direct instance in the same taint-and-sink family.

168. How does deppy compare to SSRF analysis?

Traditional approach. SSRF analysis prevents attacker-controlled URLs or network targets from reaching privileged internal services.

Deppy analogue. Deppy can express SSRF as a mismatch between a local policy section at the outbound request site and the actual provenance or validation state of the URL. Allow lists, parsers, and policy checks are then local repairs that may or may not resolve the obstruction.

Relationship. Relationship: direct policy-flow analogue. The same local-to-global pattern applies cleanly.

169. How does deppy compare to authorization logic?

Traditional approach. Authorization logics reason about principals, delegation, and whether a requested action is justified by policy statements.

Deppy analogue. Deppy is not a full authorization logic, but local authorization facts can live at call boundaries, request sites, and resource operations. The gluing perspective is appealing because one can ask whether the distributed policy evidence across layers actually composes into a coherent justification.

Relationship. Relationship: partial analogue. Good for integrating policy evidence, not for replacing dedicated authorization calculi.

170. How does deppy compare to authentication-protocol verification?

Traditional approach. Authentication protocols are usually verified by symbolic protocol models, correspondence assertions, or computational proofs.

Deppy analogue. Deppy lacks the full message algebra and adversary model of protocol verifiers. Its current mathematics could describe some local correspondence checks, but that would be an adaptation rather than the primary intended use.

Relationship. Relationship: mostly orthogonal. Protocol verification remains a more specialized and complete field here.

171. How does deppy compare to cryptographic proofs?

Traditional approach. Cryptographic proofs reason about computational hardness, adversarial advantage, and simulation-based security definitions.

Deppy analogue. Deppy does not compete with that level of mathematics. At most, it can import cryptographic guarantees as given facts or check API-level misuse around their deployment in code.

Relationship. Relationship: orthogonal. Cryptographic security proofs sit beneath or beside deppy's code-level reasoning.

172. How does deppy compare to side-channel analysis?

Traditional approach. Side-channel analysis studies leaks through timing, memory access patterns, or microarchitectural behavior rather than explicit data flow.

Deppy analogue. Deppy can express some high-level secret-dependent control-flow concerns, but true side-channel verification requires low-level cost and hardware semantics absent from its core. It is therefore better for semantic misuse than for constant-time microarchitecture arguments.

Relationship. Relationship: limited analogue. It can catch obvious semantic precursors, not full side-channel proofs.

173. How does deppy compare to secret-dependent control-flow analysis?

Traditional approach. This analysis asks whether high inputs steer branches or behaviors that should remain low-observable.

Deppy analogue. Deppy's site model is friendly to this because branch guards are explicit observation sites. A secret-dependent branch can thus become a localized relational mismatch between what low observers should and should not be able to distinguish.

Relationship. Relationship: strong local analogue. Branch sites make this one of the easier hyperproperty-style adaptations.

174. How does deppy compare to constant-time verification?

Traditional approach. Constant-time verification proves that execution cost does not depend on secret inputs, typically with cost semantics or specialized type systems.

Deppy analogue. Deppy does not have a built-in cost semantics detailed enough for true constant-time claims. It can notice obvious secret-dependent branching or dataflow structure, but that is only a coarse proxy for timing noninterference.

Relationship. Relationship: partial and weaker analogue. Constant-time remains a specialized quantitative proof discipline.

175. How does deppy compare to input-validation reasoning?

Traditional approach. Input-validation methods prove that external data is parsed, constrained, and normalized before sensitive use.

Deppy analogue. Deppy is well matched here because validators are exactly the local guards that turn weak incoming sections into stronger internal sections. When validation is missing or insufficient, the overlap between input boundary and sink requirement exposes the obstruction directly.

Relationship. Relationship: strong direct analogue. Validation is gluing repair at the boundary of trust.

176. How does deppy compare to API misuse detection?

Traditional approach. API misuse detection checks protocol order, argument validity, and lifecycle constraints around libraries and frameworks.

Deppy analogue. This is one of deppy's natural practical strengths. Calls, returns, and error sites already define a site cover over which protocol and contract facts can be transported, with misuse surfacing as unresolved gluing mismatches.

Relationship. Relationship: direct instance and good fit. API misuse becomes local protocol incoherence.

177. How does deppy compare to configuration verification?

Traditional approach. Configuration verification checks that flags, modes, and deployment choices satisfy security or correctness policies.

Deppy analogue. Deppy can treat configuration facts as sections that must align with use sites and policy sites. The payoff is that configuration weakness is not merely a static checklist item; it becomes another obstruction family explained in the same language as code bugs.

Relationship. Relationship: semantic unification. Config errors are folded into the same presheaf machinery as code-level issues.

178. How does deppy compare to supply-chain policy checking?

Traditional approach. Supply-chain policy checking validates dependency provenance, versions, signatures, and allowed build inputs against organizational rules.

Deppy analogue. Deppy is not a full supply-chain verifier, but its trust and provenance structures point in the right direction. It can record where evidence came from and how strong it is, which is exactly the metadata supply-chain reasoning cares about.

Relationship. Relationship: conceptual analogue at the evidence layer. The trust lattice is more mature than the current end-to-end supply-chain tooling.

179. How does deppy compare to secure-default reasoning?

Traditional approach. Secure-default reasoning prefers systems that are safe in the absence of extra configuration or expert action.

Deppy analogue. Deppy supports that mentality by making missing guards, missing sanitizers, and unsafe defaults appear as first-class obstructions rather than undocumented assumptions. Its reports naturally emphasize where safety depended on an absent local repair.

Relationship. Relationship: methodological alignment. The framework rewards explicit local justification and punishes silent unsafe defaults.

180. How does deppy compare to capability systems?

Traditional approach. Capability systems represent authority explicitly as unforgeable tokens, preventing many classes of ambient-authority misuse.

Deppy analogue. Deppy can check code that manipulates capability-like objects and can report mismatches in where authority is assumed to exist. But it does not itself impose a capability discipline on the language runtime.

Relationship. Relationship: weaker retrospective analogue. Capability systems prevent authority misuse by construction; deppy diagnoses it semantically in ordinary code.

181. How does deppy compare to auditability and provenance reasoning?

Traditional approach. Auditability methods care about where evidence, data, and decisions came from so that humans can trust or challenge the result appropriately.

Deppy analogue. This is one of deppy's most underappreciated strengths. The oracle monad and trust lattice attach provenance and confidence to semantic judgments, which means explanation is built into the proof object instead of stored as an afterthought.

Relationship. Relationship: strong generalization of provenance tracking. Provenance is not just logging metadata; it is part of the semantic artifact.

Chapter 10

Methodology, diagnosis, and the specifically deppy viewpoint

The final chapter compares deppy to the process-level and mathematical ideas that shape how verification is explained, maintained, and improved over time.

182. How does deppy compare to assume-guarantee reasoning?

Traditional approach. Assume-guarantee reasoning decomposes proofs by letting components assume contracts about their environment and guarantee contracts in return.

Deppy analogue. Deppy's boundary sites and local sections fit that pattern exactly, but the global question is sharper: do all the local assumptions and guarantees glue into a single coherent global section? If not, the failure is localized rather than lost inside a failed composition step.

Relationship. Relationship: strong analogue with better failure classification. Assume-guarantee clauses become gluing data.

183. How does deppy compare to abstraction-refinement loops?

Traditional approach. Abstraction-refinement loops iteratively improve a coarse model using spurious counterexamples or failed proofs as guidance.

Deppy analogue. Deppy can support that workflow, but its distinctive promise is to make the guidance more structured. An obstruction class tells you not just that the model is too coarse, but which overlaps and how many independent incompatibilities are driving the failure.

Relationship. Relationship: refinement guidance with richer diagnostics. Cohomology can inform where refinement is worth paying for.

184. How does deppy compare to bug localization?

Traditional approach. Bug localization tries to pinpoint the program statements or assumptions most responsible for an observed failure.

Deppy analogue. Deppy is almost built around localization. Each obstruction comes with disagreeing sites and overlap structure, so localization is not an after-market heuristic but a direct reflection of how local facts fail to glue.

Relationship. Relationship: strong direct analogue with a principled mathematical basis.

185. How does deppy compare to root-cause analysis?

Traditional approach. Root-cause analysis attempts to distinguish symptoms from the minimal underlying causes of a failure.

Deppy analogue. The rank story over GF(2) is precisely deppy's answer to this. A dozen warnings may collapse to two independent obstruction classes, giving a more structural notion of root cause than a flat warning list can usually provide.

Relationship. Relationship: asymptotic and conceptual improvement. Root cause becomes linear independence in obstruction space.

186. How does deppy compare to repair synthesis?

Traditional approach. Repair synthesis proposes edits that would eliminate a bug or make a proof obligation succeed.

Deppy analogue. Deppy does not automatically synthesize all repairs, but it gives unusually actionable repair coordinates: where the mismatch lives, what local fact is missing, and how many independent fixes are needed overall. That is stronger guidance than an unanalyzed failure trace.

Relationship. Relationship: diagnosis-first analogue. It is a strong precursor to repair synthesis even when not a complete synthesizer.

187. How does deppy compare to minimum hitting set diagnostics?

Traditional approach. Many debugging and diagnosis pipelines reduce explanation to minimum hitting set over conflict sets, which is expressive but typically NP-hard.

Deppy analogue. Deppy claims a genuine asymptotic advantage for one structured subclass of this problem: when conflicts are organized as first cohomology over GF(2), the minimum number of independent fixes is the rank of a binary matrix. That replaces a generic combinatorial search with linear algebra.

Relationship. Relationship: explicit asymptotic improvement. This is one of deppy's headline separation results.

188. How does deppy compare to traceability from spec to code?

Traditional approach. Traceability links requirements, proofs, tests, and code locations so humans can explain why an implementation satisfies a specification.

Deppy analogue. Deppy's layered object is naturally traceable because intent, structural checks, semantic judgments, proof evidence, and code sit in one typed artifact. Traceability is not a parallel

documentation process; it is encoded in the same semantic object that verification uses.

Relationship. Relationship: strong generalization. Traceability is internalized rather than bolted on.

189. How does deppy compare to explicit proof obligations?

Traditional approach. Formal methods often succeed by making obligations explicit so that humans and tools know exactly what must still be shown.

Deppy analogue. Deppy follows that discipline closely. Its virtue is that obligations are stratified by evidence kind and attached to local sites, so open obligations are not just pending theorems but pending coherence conditions between layers and regions.

Relationship. Relationship: direct analogue with finer structure. Open goals are localized and trust annotated.

190. How does deppy compare to the precision-versus-recall tradeoff?

Traditional approach. Static analysis routinely balances finding all bugs against avoiding false positives, and different tools pick different points on that spectrum.

Deppy analogue. Deppy is candid about operating at high recall with nontrivial false positives in some settings. The wise difference is that it tries to classify confidence and evidence strength per obstruction, rather than pretending every warning has the same epistemic status.

Relationship. Relationship: methodological improvement in reporting, not elimination of the tradeoff itself.

191. How does deppy compare to explainable verification?

Traditional approach. Explainable verification emphasizes reports that developers can act on, not just theorem statements or pass-fail results.

Deppy analogue. Deppy's mathematics is unusually hospitable to explanation because gluing failures are local by construction. A developer can be shown the required fact, the available fact, the overlap where they disagree, and the evidence supporting the claim.

Relationship. Relationship: strong direct fit. Explanation emerges from the native representation.

192. How does deppy compare to human-in-the-loop verification?

Traditional approach. Human-in-the-loop workflows let experts inspect partial results, adjust assumptions, and guide proof or analysis where automation is weak.

Deppy analogue. Deppy's trust lattice implicitly invites that workflow. Humans can decide whether an oracle judgment is good enough, whether a given theorem should be imported, or whether a low-confidence obstruction deserves more proof effort.

Relationship. Relationship: direct workflow analogue. Human judgment is integrated as a controlled component, not as an embarrassment.

193. How does deppy compare to natural-language specifications?

Traditional approach. Natural-language specifications are accessible to humans but notoriously hard to make precise or trustworthy in a proof pipeline.

Deppy analogue. Deppy confronts that tension head on. It allows natural-language intent and guarantees, but refuses to let them masquerade as proofs; they inhabit explicit low-to-mid trust layers that must still cohere with structural, semantic, and proof evidence.

Relationship. Relationship: wise hybridization. Natural language is neither banned nor blindly trusted.

194. How does deppy compare to mechanized metatheory?

Traditional approach. Mechanized metatheory proves soundness, completeness, convergence, or other framework theorems inside a proof assistant.

Deppy analogue. Deppy treats such meta-results as essential, not ornamental. The Lean development is what turns the cohomological story from metaphor into a claimed formal framework, especially for descent, exactness, and the chain-complex laws.

Relationship. Relationship: direct alignment. Mechanized metatheory is the credibility backbone of deppy's mathematical claims.

195. How does deppy compare to category-theoretic semantics?

Traditional approach. Category theory often appears in semantics as a unifying language for composition, effects, and type structure, though many practical tools stop short of using it computationally.

Deppy analogue. Deppy tries to make category theory computational rather than merely explanatory. Site categories, presheaves, monads, and exact sequences are not retrospective metaphors; they are advertised as the actual organizing algorithms.

Relationship. Relationship: ambitious operationalization. It is category-theoretic semantics used as engineering method, not only as conceptual gloss.

196. How does deppy compare to sheaf models in semantics?

Traditional approach. Sheaf models have appeared before in semantics and logic, usually to interpret contextuality, local consistency, or forcing-like constructions.

Deppy analogue. Deppy stands closest to that tradition among all its analogues. Its novelty is to make the sheaf idea concrete for program analysis tasks such as bug finding, equivalence, and incremental recomputation rather than leaving it as abstract semantic background.

Relationship. Relationship: direct descendant with an applied emphasis. This is one of the most faithful analogies in the document.

197. How does deppy compare to obstruction theory?

Traditional approach. Obstruction theory studies when local data can be extended globally and identifies cohomological classes witnessing the failure.

Deppy analogue. This is not just an analogy for deppy; it is the core mathematical identity. Bugs, inequivalences, and coherence failures are literally presented as obstruction classes rather than being merely compared to them poetically.

Relationship. Relationship: exact identity. Obstruction theory is the deppy lens, not a neighboring method.

198. How does deppy compare to product constructions that combine analyses?

Traditional approach. Classical analyzer design often builds reduced products or combinations of analyses so that multiple abstract domains inform each other.

Deppy analogue. Deppy shares that integrative instinct, but the product is geometric and evidential as well as abstract-domain based. Sites can carry multiple interacting local facts, and the layered product over intent, structure, semantics, proof, and code is itself a first-class object.

Relationship. Relationship: broad generalization. Reduced products are one local combination mechanism inside a larger layered product site.

199. How does deppy compare to incremental recomputation in practice?

Traditional approach. Incremental recomputation is usually an engineering problem of dependency tracking, caching, and invalidation after edits.

Deppy analogue. Deppy tries to give one slice of that engineering problem a theorem-shaped answer. When the local facts are organized over overlapping pieces, Mayer-Vietoris says how to recompute the global obstruction picture exactly from changed parts and boundary interaction terms.

Relationship. Relationship: conceptual and exactness improvement at a specific layer of the problem. The theorem does not remove all engineering, but it sharpens what can be reused exactly.

200. How does deppy compare to benchmark-driven evaluation?

Traditional approach. Most verification tools justify themselves empirically by running on benchmarks and comparing recall, precision, automation, or proof burden.

Deppy analogue. Deppy does that too, but its benchmarks are tied to explicit mathematical claims: recall for bug obstruction detection, exactness for equivalence, and independent-fix counting

via first-cohomology rank. The empirical story is meant to test a theory, not just a heuristic pipeline.

Relationship. Relationship: conventional evaluation with unusually theory-linked metrics.

Closing note

The most useful way to read the catalogue is not as a claim that deppy replaces the entire formal-methods ecosystem. Its strongest identity is as a local-to-global organizer: it absorbs contracts, solver results, proof artifacts, runtime evidence, and semantic judgments into a common object, then uses gluing and obstruction theory to say what still does not fit.

That is why some comparisons in the document are marked as strict generalizations, others as partial analogues, and others as orthogonal. The point of being wise about the analogue is exactly to preserve those distinctions.